



Stack technique

Architecture, technologies, hébergement, sécurité et configuration
Document d'architecture technique

Version 1.0 · Application v1.0.0+1

Rédigé le 28 août 2026

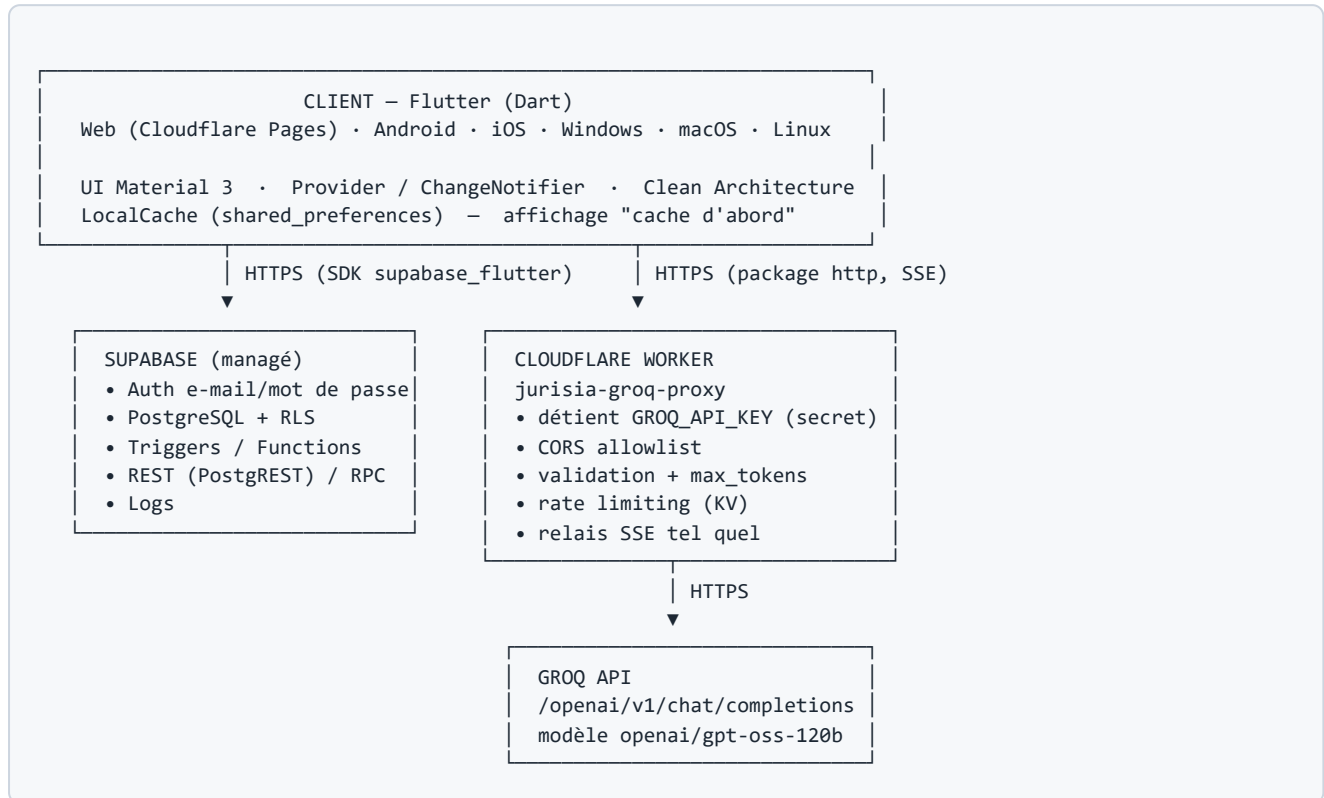
Flutter 3.44.6 · Dart 3.12.2 · Supabase · Cloudflare Workers · Groq

Sommaire

1. Vue d'ensemble de l'architecture
 2. Application cliente (Flutter)
 - 2.1 Socle et plateformes cibles
 - 2.2 Dépendances
 - 2.3 Polices et ressources
 - 2.4 Architecture logicielle
 - 2.5 Gestion d'état
 - 2.6 Cache local
 - 2.7 Thème et design system
 3. Backend & services managés
 - 3.1 Supabase (Auth + PostgreSQL + RLS)
 - 3.2 Relais IA — Cloudflare Worker
 - 3.3 API Groq (modèle de langage)
 4. Intégration IA — flux technique
 5. Modèle de données (SQL)
 6. Hébergement & déploiement
 7. Intégration continue (CI/CD)
 8. Configuration, variables et secrets
 9. Sécurité technique
 10. Observabilité et exploitation
 11. Qualité, tests et outils
 12. Arborescence du dépôt
 13. Environnement de développement
 14. Limites connues et dette technique
- Annexe — Tableau récapitulatif des technologies

1. Vue d'ensemble de l'architecture

JurisIA est une application Flutter unique (une base de code, plusieurs cibles) qui s'appuie sur deux services managés indépendants : **Supabase** pour l'authentification et la persistance, et un **Cloudflare Worker** qui relaie les appels au modèle de langage **Groq** tout en détenant la clé API côté serveur.



Principes directeurs :

- Le client ne détient **aucun secret d'IA** ; il ne connaît que l'URL publique du relais.
- L'URL et la clé publique Supabase **ne sont pas des secrets** (conçues pour être embarquées) ; la sécurité des données repose sur la RLS.
- Chaque service peut tomber sans faire planter l'application (dégradation gracieuse).
- Frontières abstraites côté client : le fournisseur d'IA et les sources de données sont substituables.

2. Application cliente (Flutter)

2.1 Socle et plateformes cibles

Élément	Valeur
Framework	Flutter 3.44.6 (canal stable)
Langage	Dart 3.12.2 (SDK contrainte <code>^3.12.2</code>)
Design system	Material 3 (<code>useMaterial3: true</code>), thème sombre unique
Nom du paquet	<code>jurisia_app</code> — version <code>1.0.0+1</code> — <code>publish_to: none</code>
Rendu web	CanvasKit / Skwasm (build web Flutter standard)
Cibles	Web, Android, iOS, Windows, macOS, Linux (dossiers plateforme présents)
Icônes natives	Générées pour les plateformes

État de publication : **Web en production** (Cloudflare Pages) ; **Android** testé en build de debug ; **iOS** non compilé (nécessite un Mac) ; **desktop** supporté par le code.

2.2 Dépendances

Dépendances de production (`pubspec.yaml`)

Paquet	Version	Rôle dans JurisIA
<code>flutter</code>	SDK	Framework UI
<code>supabase_flutter</code>	<code>^2.17.2</code>	Authentification (e-mail / mot de passe), client PostgREST, RPC, flux d'état d'auth
<code>http</code>	<code>^1.2.2</code>	Client HTTP pour l'appel streaming (SSE) au relais IA
<code>provider</code>	<code>^6.1.2</code>	Injection de dépendances légère et gestion d'état (contrôleurs <code>ChangeNotifier</code>)
<code>shared_preferences</code>	<code>^2.3.2</code>	Cache local : largeur / état de la sidebar, historique et profil « cache d'abord »
<code>equatable</code>	<code>^2.0.5</code>	Égalité de valeur des entités <code>domain</code> / <code>data</code> sans boilerplate
<code>uuid</code>	<code>^4.5.1</code>	Identifiants de conversations, messages, résultats, évaluations...
<code>cupertino_icons</code>	<code>^1.0.8</code>	Jeu d'icônes iOS

Dépendances de développement

Paquet	Version	Rôle
<code>flutter_test</code>	SDK	Tests unitaires et de widgets
<code>flutter_lints</code>	<code>^6.0.0</code>	Règles de lint recommandées (via <code>analysis_options.yaml</code>)

Pile volontairement minimale : pas de générateur de code, pas de router tiers, pas de client GraphQL, pas de `google_fonts` au runtime (les polices sont embarquées). Cela réduit la surface de dépendances et les temps de build.

2.3 Polices et ressources

- **Libre Caslon Display** (Regular) — titres serif.
- **Public Sans** (Regular / Medium 500 / SemiBold 600 / Bold 700) — corps de texte.
- **Lora** (Regular) — usage éditorial.
- Fichiers `.ttf` dans `assets/fonts/`, déclarés dans `pubspec.yaml` — embarqués dans le build (aucune requête CDN au runtime, y compris sur le Web).
- Icônes : Material Icons (`uses-material-design: true`) + Cupertino Icons.

2.4 Architecture logicielle

Organisation par **fonctionnalité** (`lib/features/<module>/`), chacune en Clean Architecture à trois couches :

Couche	Contenu	Exemples
<code>domain</code>	Entités, contrats de repository, use cases (pas de dépendance framework)	<code>LegalAnalysisGrid</code> , <code>AnalyzeLitigationUseCase</code> , <code>GradeEvaluationUseCase</code>
<code>data</code>	Implémentations de repository, datasources (locales et Supabase), prompts système	<code>SupabaseLitigationConversationStore</code> , <code>LocalStudentCurriculumDataSource</code> , <code>ProfessionalSystemPrompt</code>
<code>presentation</code>	Écrans, widgets, contrôleurs <code>ChangeNotifier</code> , « providers » de câblage	<code>LitigationChatController</code> , <code>LibraryScreen</code>

Dossiers transverses :

- `lib/core/` — `ai/` (datasource LLM, config, découpage de flux, exceptions), `supabase/` , `auth/` (porte d'auth), `navigation/` , `shell/` (sidebar unifiée, sections contextuelles, carte profil), `storage/` (`LocalCache`), `validation/` (`AppInputLimits`), `platform/` , `legal/` (CGU / confidentialité), `widgets/` (bibliothèque de composants « premium »).
- `lib/models/` — modèles partagés : `chat/` (conversation, message), `legal_document/` (document, branche du droit), `student/` (module de cours, évaluation, progression, niveau).
- `lib/theme/` — `app_theme.dart` (palette, dégradés, rayons, espacements, thème Material).
- `lib/main.dart` — point d'entrée : initialisation parallèle (Supabase + cache), style d'overlay système, `MaterialApp` → splash → porte d'auth.

Les 143 fichiers Dart de `lib/` se répartissent sur les 5 modules + le socle `core` + les modèles partagés.

2.5 Gestion d'état

- **Provider + `ChangeNotifier`** : un contrôleur par domaine d'écran (`LitigationChatController` , `LibraryController` , `StudentController` , `ProfessionalDocumentsController` , `ContactProfessionalController` , `ProfileController` , plus des contrôleurs de sous-écran : `ModuleTutorController` , `EvaluationController` , `DraftingWorkspaceController`).
- Les contrôleurs partagés sont fournis par un `MultiProvider` au niveau de la coquille (`AppShell`) pour survivre à la navigation entre espaces.

- `AppShellScope` (`InheritedWidget`) expose le contrôle de navigation (changer d'espace, replier la nav) à tous les écrans.
- Câblage des dépendances : fonctions `build<X>Controller()` par module (fichiers `*_providers.dart`) qui assemblent datasources + repositories + use cases.

2.6 Cache local

- `LocalCache` : surcouche mono-instance de `shared_preferences` (localStorage sur le Web, fichier natif ailleurs), initialisée une fois au démarrage pour offrir des **lectures synchrones**.
- Usages : largeur et état replié de la sidebar ; dernier historique de consultations connu (par utilisateur) ; dernier profil connu.
- Tout est au mieux effort : si le stockage est indisponible (navigation privée stricte, test sans binding), l'application fonctionne sans affichage instantané.

2.7 Thème et design system

- Thème Material 3 sombre unique (`ThemeMode.dark`), construit dans `AppTheme.darkTheme` .
- Jetons centralisés : `AppColors` , `AppGradients` , `AppRadius` , `AppSpacing` (grille 4 px), `AppLetterSpacing` , `AppShadows` .
- Transition de page personnalisée (`_LuxuryFadeThroughTransitionsBuilder` , 320 ms) appliquée à toutes les plateformes.
- Thématisation exhaustive des composants (AppBar, Card, Chip, Dialog, SnackBar, TabBar, NavigationRail, inputs, switches, tooltips...).

3. Backend & services managés

3.1 Supabase (Auth + PostgreSQL + RLS)

Aspect	Détail
Authentification	E-mail / mot de passe (provider par défaut). Métadonnées utilisateur : <code>full_name</code> , <code>profession</code> . Confirmation d'e-mail configurable côté projet.
Base	PostgreSQL managé. 9 tables applicatives (voir §5). RLS activée sur toutes.
Accès données	SDK <code>supabase_flutter</code> → PostgREST (<code>from().select()/upsert()/insert()/update()/delete()</code>) et RPC (<code>rpc('increment_download_count')</code>).
Logique serveur	Fonction <code>handle_new_user()</code> + trigger <code>on_auth_user_created</code> (création de profil) ; fonction <code>increment_download_count(doc_id)</code> (upsert atomique).
Clés	URL de projet + « anon key » (publishable key) — non secrètes , embarquées dans le client. Fournies via <code>--dart-define</code> , avec valeurs par défaut compilées.
Migrations	<code>server/supabase/schema.sql</code> puis <code>migration_002 ... migration_005</code> (idempotentes), exécutées manuellement dans l'éditeur SQL du tableau de bord.
Logs	Journaux Auth / API / base dans le tableau de bord Supabase.

Stratégie de persistance côté client : chaque module tient son état en mémoire pour la réactivité, puis synchronise en arrière-plan (« au mieux effort ») quand un `SupabaseClient` et un `userId` sont disponibles. Une méthode `hydrate()` recharge l'état au démarrage. Exception : le module 05 exige la connexion (échec explicite sinon).

3.2 Relais IA — Cloudflare Worker

Aspect	Détail
Nom	<code>jurisia-groq-proxy</code> — <code>src/index.js</code> (Worker ES module)
Runtime	Cloudflare Workers (V8 isolates) — <code>compatibility_date = 2026-08-25</code>
Outil	Wrangler <code>^4.126.0</code> (<code>npm run dev</code> / <code>npm run deploy</code>)
Endpoint exposé	<code>POST /v1/chat/completions</code> (compatible OpenAI) ; 404 sur tout autre chemin ; 405 hors POST
Secret	<code>GROQ_API_KEY</code> — <code>wrangler secret put</code> , jamais dans un fichier versionné
Binding KV	<code>RATE_LIMIT_KV</code> — compteurs de limitation de débit
Observabilité	<code>[observability] enabled = true</code> dans <code>wrangler.toml</code> (Workers Logs)

Contrôles appliqués par le relais

- **CORS** : origine reflétée seulement si elle appartient à la liste (`localhost` , `127.0.0.1` , `^https://([a-z0-9-]+\.)?jurisia-app\.pages\.dev$`). Absence d'en-tête `Origin` = client natif → accepté. Origine hors liste → `403` .

- **Modèle** : rejet si le modèle demandé n'est pas dans `ALLOWED_MODELS` (`openai/gpt-oss-120b`) — à tenir synchronisé avec le client.
- **Taille** : corps > 60 000 octets → `413` ; contenu cumulé des messages > 24 000 caractères → `400` .
- **Tokens** : `max_tokens` plafonné silencieusement à 4096.
- **Débit** : 20 requêtes / minute et 300 / jour par IP (`CF-Connecting-IP`), compteurs KV avec TTL. `429` au dépassement.
- **Streaming** : le corps de la réponse Groq (flux SSE) est relayé **tel quel, sans tampon** ; les en-têtes CORS sont ajoutés à la réponse.
- **Journal** : événements `completed` / `rate_limited` / `rejected` / `upstream_error` / `misconfigured` en JSON structuré — **jamais le contenu des messages**.

Limite connue : les compteurs KV ne sont pas fortement cohérents ni atomiques — deux requêtes concurrentes de la même IP peuvent lire l'ancien compteur. Frein sérieux contre l'abus grossier, pas une garantie exacte (un Durable Object serait nécessaire). Le relais n'authentifie pas encore l'appelant par compte.

3.3 API Groq (modèle de langage)

Aspect	Détail
Endpoint amont	<code>https://api.groq.com/openai/v1/chat/completions</code>
Modèle	<code>openai/gpt-oss-120b</code> — modèle « raisonneur » (budget de raisonnement interne non visible avant la réponse)
Mode	<code>stream: true</code> (SSE), format de messages <code>{role, content}</code>
Authentification	<code>Authorization: Bearer <GROQ_API_KEY></code> — ajouté uniquement par le Worker
Historique	La famille Llama 3.x (précédemment utilisée) a été retirée du catalogue Groq (404 <code>model_not_found</code> , vérifié le 25/08/2026) ; migration vers <code>openai/gpt-oss-120b</code> . Le projet a auparavant migré d'Anthropic Claude vers Groq.

4. Intégration IA — flux technique

4.1 Chaîne d'appel côté client

```
Contrôleur d'écran
→ UseCase (domain)
  → RepositoryImpl (data)      construit system prompt + messages [{role, content}]
    → LlmDataSource (abstraction)
      → GroqDataSource          http.Request POST, body { model, messages, max_tokens,
stream:true }
        → Relais Cloudflare     valide, borne, relaie
          → Groq API             renvoie un flux SSE
            ← flux SSE relayé tel quel
              ← Stream<String>   fragments de texte (delta.content), 'data:' / '[DONE]'
                ← Stream de "chunks" métier (texte visible, done, grille/risques)
                  ← état d'écran mis à jour au fil de l'eau
```

4.2 Séparation du texte visible et du bloc caché

`HiddenBlockStreamSplitter` : découpe le flux reçu fragment par fragment en (a) une partie **visible** — tout ce qui précède le premier marqueur d'ouverture — et (b) un **bloc caché** accumulé à partir du marqueur. Il gère le cas où le marqueur lui-même est coupé entre deux paquets du flux. Utilisé par :

- Module 01 — marqueur `<<<JURISIA_GRID_JSON>>>` → grille d'analyse (`LegalAnalysisGrid.fromJson`), + branche du droit + complexité.
- Module 04 (audit) — marqueur `<<<JURISIA_RISKS_JSON>>>` → tableau `ClauseRisk`.

Un JSON invalide dans le bloc caché est ignoré (`FormatException` / `TypeError` capturées) : l'état précédent est conservé, l'échange n'est pas interrompu.

4.3 Construction des messages

- Le message système est **statique** (persona + protocole). Le contenu utilisateur part toujours en message `role: user` — **jamais interpolé dans le système** (défense anti-injection ; le prompt professionnel l'énonce explicitement au modèle).
- Module 01 : l'historique `user` / `assistant` est transmis depuis le premier message utilisateur ; un rappel interne du contenu déjà connu de la grille est ajouté au système pour éviter les redemandes.
- Module 04 : les 3 documents les plus pertinents de la bibliothèque (par branche) sont formatés en appui dans le système et restitués comme sources citées.
- Module 03 : les leçons du module sont injectées dans le prompt du tuteur et du générateur d'évaluations ; ce dernier exige une sortie strictement JSON.

4.4 Génération de titre de consultation

Prompt système fixe (3–6 mots), `max_tokens: 200` — dimensionné pour laisser la place au titre après le budget de raisonnement du modèle. Nettoyage : suppression des guillemets et du point final.

5. Modèle de données (SQL)

```
-- Extrait représentatif (server/supabase/schema.sql + migrations 002-005)

create table public.profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  display_name text, full_name text, profession text,
  terms_accepted_at timestamptz,
  created_at timestamptz not null default now()
);
alter table public.profiles enable row level security;
-- policies : select/update/insert where auth.uid() = id

create table public.litigation_conversations (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users(id) on delete cascade,
  title text not null default 'Nouvelle consultation',
  domain text, complexity text,
  analysis_grid jsonb,
  is_favorite boolean not null default false,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);
create table public.litigation_messages (
  id uuid primary key default gen_random_uuid(),
  conversation_id uuid not null
    references public.litigation_conversations(id) on delete cascade,
  sender text not null check (sender in ('user','assistant')),
  content text not null,
  suggested_professional text,
  created_at timestamptz not null default now()
);
create table public.library_favorites (
  user_id uuid not null references auth.users(id) on delete cascade,
  document_id text not null,
  created_at timestamptz not null default now(),
  primary key (user_id, document_id)
);
create table public.library_document_stats (
  document_id text primary key,
  download_count integer not null default 0
);
create table public.student_module_progress (
  user_id uuid not null references auth.users(id) on delete cascade,
  module_id text not null,
  is_unlocked boolean not null default false,
  is_completed boolean not null default false,
```

```

    best_score numeric,
    attempts_count integer not null default 0,
    updated_at timestamptz not null default now(),
    primary key (user_id, module_id)
);

create table public.student_evaluation_attempts (
    id uuid primary key default gen_random_uuid(),
    user_id uuid not null references auth.users(id) on delete cascade,
    module_id text not null,
    attempt_number integer not null,
    score numeric not null,
    created_at timestamptz not null default now()
);

create table public.professional_drafting_results (
    id uuid primary key default gen_random_uuid(),
    user_id uuid not null references auth.users(id) on delete cascade,
    mode text not null check (mode in ('redaction','audit','consultation')),
    title text not null, content text not null,
    risks jsonb not null default '[]'::jsonb,
    cited_sources jsonb not null default '[]'::jsonb,
    is_favorite boolean not null default false,
    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now()
);

create table public.professional_contact_requests (
    id uuid primary key default gen_random_uuid(),
    user_id uuid not null references auth.users(id) on delete cascade,
    category text not null check (category in
        ('notaire','avocat','juriste','huissier','greffier','juge')),
    full_name text not null, contact_info text not null, message text not null,
    status text not null default 'pending'
        check (status in ('pending','contacted','closed')),
    created_at timestamptz not null default now()
);

-- Toutes les tables : RLS activée, policy "auth.uid() = user_id"
-- (sauf library_document_stats : lecture publique, écriture authentifiée)

```

Types applicatifs côté Dart : branches du droit (13), types de document (8), niveaux académiques (5), modes professionnels (3), niveaux de risque (3), catégories de contact (6), professions utilisateur (7).

6. Hébergement & déploiement

6.1 Application web — Cloudflare Pages

```
flutter build web --release
npx wrangler pages deploy build/web --project-name=jurisia-app
```

- URL de production : `https://jurisia-app.pages.dev` (plan gratuit).
- Chaque déploiement obtient aussi une URL de prévisualisation `https://<hash>.jurisia-app.pages.dev` — les deux formes sont déjà autorisées par la liste CORS du relais.
- Un domaine personnalisé futur devra être ajouté à `ALLOWED_ORIGIN_PATTERNS` puis le Worker redéployé.

6.2 Relais IA — Cloudflare Workers

```
# Une seule fois, depuis server/groq-proxy/
npm install
npx wrangler login
npx wrangler secret put GROQ_API_KEY # colle la clé gsk...
npx wrangler kv namespace create RATE_LIMIT_KV # puis reporter l'id dans wrangler.toml
npx wrangler deploy

# Après toute modification de src/index.js ou wrangler.toml
npx wrangler deploy
```

6.3 Backend — Supabase

- Créer un projet Supabase (niveau gratuit).
- Exécuter `schema.sql` puis `migration_002 ... 005` dans l'éditeur SQL.
- Récupérer l'URL de projet et la clé publique (Project Settings → Data API).
- Fournir ces valeurs au client via `--dart-define` (ou laisser les valeurs par défaut compilées).

6.4 Mobile & desktop

```
flutter build apk --debug # Android (testé)
flutter build apk --release # Android (non encore publié)
flutter build windows / macos / linux
flutter build ios # nécessite un Mac (non réalisé)
```

7. Intégration continue (CI/CD)

Fichier `.github/workflows/ci.yml` — déclenché sur chaque `push` et `pull_request` vers `main`. Version Flutter épinglée : `3.44.6` (canal stable).

Job	Runner	Étapes
<code>analyze-and-test</code>	ubuntu-latest	<code>flutter pub get</code> → <code>flutter analyze</code> → <code>flutter test</code>
<code>build-android</code>	ubuntu-latest (Java 17 Temurin)	<code>flutter build apk --debug</code> — dépend du job précédent
<code>build-web</code>	ubuntu-latest	<code>flutter build web</code> — dépend du job précédent

iOS / macOS / Windows sont volontairement absents (coût des runners GitHub) — à ajouter sur un plan payant ou si ces plateformes deviennent prioritaires. Aucun déploiement automatique : la publication web et Worker est manuelle (wrangler).

8. Configuration, variables et secrets

8.1 Paramètres de compilation du client (`--dart-define`)

Clé	Défaut compilé	Nature
<code>GROQ_PROXY_URL</code>	<code>https://jurisia-groq-proxy.jurisia-api.workers.dev/v1/chat/completions</code>	URL publique (non secrète)
<code>GROQ_MODEL</code>	<code>openai/gpt-oss-120b</code>	Non secret
<code>SUPABASE_URL</code>	<code>https://gfpguuuzzyqoxjkhli.supabase.co</code>	Non secret (conçu pour être embarqué)
<code>SUPABASE_ANON_KEY</code>	<code>sb_publishable_...</code>	Non secret (clé publique ; RLS protège les données)

```
flutter run -d chrome \  
  --dart-define=GROQ_PROXY_URL=https://<worker>/v1/chat/completions \  
  --dart-define=SUPABASE_URL=https://<projet>.supabase.co \  
  --dart-define=SUPABASE_ANON_KEY=<anon_key>
```

8.2 Secrets (jamais versionnés)

Secret	Emplacement	Gestion
<code>GROQ_API_KEY</code> (<code>gsk_...</code>)	Cloudflare Worker	<code>wrangler secret put GROQ_API_KEY</code>
Mots de passe utilisateurs	Supabase Auth	Gérés par Supabase (jamais vus par l'app)

Ce qui n'est **pas** un secret : l'URL du relais, l'URL et la clé publique Supabase. Elles peuvent être committées. La seule valeur réellement secrète est la clé API Groq, isolée dans le Worker.

8.3 Configuration du Worker (`wrangler.toml`)

```
name = "jurisia-groq-proxy"  
main = "src/index.js"  
compatibility_date = "2026-08-25"  
  
[[kv_namespaces]]  
binding = "RATE_LIMIT_KV"  
id = "<id du namespace KV>"  
  
[observability]  
enabled = true
```

9. Sécurité technique

Couche	Mesure
Clé d'IA	Détenue par le Worker (secret Cloudflare). Jamais dans le binaire client ni dans le trafic réseau du client. Ajoutée à la requête uniquement côté Worker.
Accès aux données	RLS PostgreSQL sur toutes les tables : <code>auth.uid() = user_id</code> . Fonctions <code>security_definer</code> / <code>security invoker</code> avec <code>search_path</code> figé.
Suppression de compte	<code>on delete cascade</code> depuis <code>auth.users</code> → toutes les données liées sont purgées.
Relais — entrée	Validation du modèle, de la taille du corps (60 Ko), du volume de messages (24 000 car.), plafond <code>max_tokens</code> (4096).
Relais — origine	Liste d'origines CORS explicite ; reflet de l'origine seulement si autorisée (pas de <code>*</code>) ; clients natifs sans <code>Origin</code> acceptés.
Relais — débit	20/min et 300/jour par IP (KV). Possibilité d'ajouter une règle WAF native Cloudflare en complément.
Prompt IA	Contenu utilisateur jamais interpolé dans le prompt système ; consigne explicite anti-injection ; consigne anti-hallucination.
Journalisation	Aucun contenu de message journalisé (client et Worker) ; métadonnées uniquement.
Transport	HTTPS de bout en bout (Pages, Workers, Supabase, Groq).

Points ouverts (feuille de route sécurité)

- Authentification de l'appelant au niveau du Worker (jeton utilisateur vérifié) → limitation par compte plutôt que par IP.
- Limitation de débit exacte (Durable Object) si l'abus concurrent devient un problème.
- Revue juridique des CGU / politique de confidentialité.

10. Observabilité et exploitation

Service	Moyens
Relais Worker	<code>wrangler tail</code> (temps réel) ; Workers Logs (rétention quelques jours, filtrable) ; onglet Metrics (requêtes, erreurs, latence, CPU) ; Notifications Cloudflare (alerte e-mail sur taux d'erreur / volume).
Événements journalisés	<code>completed</code> (modèle, statut, durée) · <code>rate_limited</code> · <code>rejected</code> (raison) · <code>upstream_error</code> · <code>misconfigured</code> .
Supabase	Journaux Auth / API / base dans le tableau de bord (aucune config supplémentaire).
Groq	Consommation et quotas sur <code>console.groq.com</code> .
Client (dev)	Signalement <code>FlutterError.reportError</code> si le schéma Supabase est désynchronisé (codes PostgreSQL 42703 / 42P01 / PGRST204 / PGRST205).

11. Qualité, tests et outils

- **Analyse statique :** `flutter analyze` avec `package:flutter_lints/flutter.yaml` (`analysis_options.yaml`).
- **Tests :** `flutter test` — 9 fichiers de test :
 - `litigation_repository_test.dart`, `litigation_history_test.dart`
 - `library_repository_test.dart`
 - `student_repository_test.dart`
 - `professional_repository_test.dart`
 - `contact_professional_test.dart`
 - `hidden_block_stream_splitter_test.dart` (découpage de flux à bloc caché)
 - `profile_controller_test.dart`
 - `widget_test.dart` (navigation : sidebar tiroir / permanente, feuille profil, absence de débordement par espace)
- **Équivalence de valeur :** `equatable` sur les entités facilite les assertions de test.
- **Testabilité :** datasources et repositories injectables ; `SupabaseConfig.isReady` / `LocalCache.instance` nul-safe hors initialisation (tests de widgets sans binding réseau).

12. Arborescence du dépôt

```
jurisia_app/
├─ lib/
│  ├─ main.dart          point d'entrée
│  ├─ core/
│     ├─ ai/             LlmDataSource, GroqDataSource, config, splitter, exceptions
│     ├─ auth/           AuthGate
│     ├─ supabase/       SupabaseConfig
│     ├─ storage/        LocalCache
│     ├─ navigation/     shell/ sidebar unifiée, sections contextuelles, profil
│     ├─ validation/     AppInputLimits
│     ├─ platform/       legal/ style plateforme, CGU / confidentialité
│     └─ widgets/        bibliothèque de composants "premium"
│  ├─ models/            chat, legal_document, student (modèles partagés)
│  ├─ features/
│     ├─ auth/           connexion / inscription
│     ├─ profile/        profil utilisateur
│     ├─ litigation/     module 01
│     ├─ library/        module 02
│     ├─ student/        module 03
│     ├─ professional/   module 04
│     └─ contact_professional/ module 05
│  └─ theme/             app_theme.dart
├─ assets/fonts/         Libre Caslon Display, Public Sans, Lora
├─ server/
│  ├─ groq-proxy/        Cloudflare Worker (src/index.js, wrangler.toml, package.json)
│  └─ supabase/          schema.sql, migration_002..005, README
├─ test/                 9 fichiers de test
├─ android/ ios/ web/ windows/ macos/ linux/  projets plateforme
├─ .github/workflows/ci.yml  CI (analyze, test, build android/web)
├─ analysis_options.yaml
├─ pubspec.yaml
└─ README.md             documentation projet et déploiement
```

13. Environnement de développement

Outil	Version / commande
Flutter	3.44.6 stable (Framework rev ee80f08bbf, Engine 83675ed276)
Dart	3.12.2
DevTools	2.57.0
Node.js	pour Wrangler (relais) — <code>npm install</code> dans <code>server/groq-proxy/</code>
Wrangler	^4.126.0

```
# Développement local
flutter pub get
flutter run -d chrome          # ou -d windows, -d <device Android>

# Qualité
flutter analyze
flutter test

# Relais en local
cd server/groq-proxy && npm install && npm run dev
```

14. Limites connues et dette technique

- Le relais IA n'authentifie pas l'appelant par compte (limitation par IP, compteurs KV non atomiques).
- La liste des modèles autorisés du Worker doit être tenue synchronisée manuellement avec `GroqApiConfig.model`.
- Catalogue de la bibliothèque juridique : base de démonstration, références illustratives.
- Le détail des questions d'une tentative d'évaluation n'est pas persisté (seuls déblocage, validation, meilleure note, nombre de tentatives le sont).
- Pas de déploiement automatisé (web / Worker déployés manuellement).
- Pas de build iOS (absence de Mac) ; pas de publication sur les stores.
- CI limitée à Ubuntu (pas de build iOS / macOS / Windows).
- Pièces jointes du chat : champ présent dans le modèle, non exploité par l'UI.
- Migrations Supabase à appliquer manuellement dans l'éditeur SQL (pas de `supabase db push` automatisé en CI).
- CGU / politique de confidentialité non relues par un juriste.

Annexe — Tableau récapitulatif des technologies

Domaine	Technologie	Version	Rôle
Client	Flutter	3.44.6	Framework applicatif multiplateforme
Langage	Dart	3.12.2	Langage du client
UI	Material 3	—	Système de composants (thème sombre custom)
État / DI	provider	^6.1.2	ChangeNotifier + injection
Backend	Supabase (supabase_flutter)	^2.17.2	Auth e-mail/mot de passe, PostgreSQL, RLS, RPC, logs
Base de données	PostgreSQL (managé Supabase)	—	9 tables applicatives + triggers + fonctions
Cache local	shared_preferences	^2.3.2	localStorage / fichier natif — affichage cache d'abord
HTTP	http	^1.2.2	Appel streaming (SSE) au relais
Utilitaires	equatable / uuid	^2.0.5 / ^4.5.1	Égalité de valeur / identifiants
Relais IA	Cloudflare Workers + Wrangler	compat. 2026-08-25 / wrangler ^4.126.0	Détient la clé Groq, valide, borne, limite le débit, relaie le SSE
Stockage relais	Cloudflare KV	—	Compteurs de limitation de débit
Modèle de langage	Groq — openai/gpt-oss-120b	—	Complétion de texte en streaming (API compatible OpenAI)
Hébergement web	Cloudflare Pages	—	jurisia-app.pages.dev (+ prévisualisations)
CI	GitHub Actions	—	analyze + test + build Android(debug) + build Web
Lint	flutter_lints	^6.0.0	Règles d'analyse statique
Polices	Libre Caslon Display, Public Sans, Lora	embarquées	Typographie (aucune dépendance CDN runtime)